P3 - 2022 Exam: Bus fare calculator (database TravelPrice)

25 marks. Pure logic + SQLite: the GUI part (4 marks) is gone from the course. Files: `Solutions_py/2022_bus_fare_travelprice_solution.py` *(window) and* `2022_bus_fare_console.py` *(terminal, practise this).*

1. What the paper asked

Requirement	Marks
Screen layout (age entry, Save Age, Clear All, two list windows, cost window)	4
Save stores each passenger's **age AND ticket price** in database **TravelPrice**, then clears the entry box	6 + 1
Display age: all saved ages	3
Display ticket price: price per age, **each price must match its age**	3
Display cost: total for the group	5
Clear All: empty the three display windows AND delete everything from the DB	2

Price rule: age 0-2 = £0; 3-12 = £7; 66 and over = £13; everyone else (13-65) = £18. **Sample to reproduce:** ages 2, 78, 68, 45, 2 -> prices 0, 13, 13, 18, 0 -> total **44**.

2. Analogy

A bus conductor with a **ticket book**. For each passenger: ask the age, work out the fare from the rules, write *both* on one line of the book (one table row). Total = add up the fare column. Clear All = tear out all the pages.

Key design decision: store age and price **in the same row**. Then "each price corresponds to its age" is automatic, because you read them back together in the same order (`ORDER BY id`).

3. Line by line (terminal version)

```
import sqlite3

conn = sqlite3.connect("TravelPrice.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS Passengers (
                      id    INTEGER PRIMARY KEY AUTOINCREMENT,
                      age   INTEGER NOT NULL,
                      price INTEGER NOT NULL)""")
conn.commit()
```

```
def ticket_price(age):
    if age <= 2:
        return 0
    elif age <= 12:
        return 7
    elif age >= 66:
        return 13
    else:
        return 18
```

- An `if / elif / else` ladder is read **top to bottom and stops at the first true test.** So by the time we reach `elif age <= 12` we already know age is 3 or more. That is why you do not need to write `3 <= age <= 12`.
- The order `<= 2`, `<= 12`, `>= 66`, else is the shortest correct way. Boundary tests to try by hand: 2 -> 0, 3 -> 7, 12 -> 7, 13 -> 18, 65 -> 18, 66 -> 13.

```
def enter_ages():
    """Keep asking for ages until the user types q."""
    while True:
        text = input("Enter the client's age (q to stop): ").strip()
        if text.lower() == "q":
            break
        try:
            age = int(text)
        except ValueError:
            print("Please enter a whole number.")
            continue
        if age < 0 or age > 120:
            print("Age must be between 0 and 120.")
            continue
        cursor.execute("INSERT INTO Passengers (age, price) VALUES (?, ?)",
                       (age, ticket_price(age)))
        conn.commit()
```

- `while True:` loop forever; `break` leaves when the user types `q`. Pattern for "enter items one at a time until I say stop".
- `text.lower() == "q"`: accept `Q` as well.
- `try: int(text) / except ValueError:` stops the program crashing when someone types `abc`. `continue` jumps back to the top of the loop for another go.
- `(age, ticket_price(age))`: calls the fare function and stores its answer next to the age.

```
def show_ages_and_prices():
    cursor.execute("SELECT age, price FROM Passengers ORDER BY id")
    print("Client's age in years | Price per client (GBP)")
    for age, price in cursor.fetchall():
        print(f"{age:>21} | {price:>6}")


def show_total():
    cursor.execute("SELECT SUM(price) FROM Passengers")
    total = cursor.fetchone()[0] or 0            # None -> 0 when empty
    print(f"Total cost for the journey: GBP {total}")


def clear_all():
    cursor.execute("DELETE FROM Passengers")
    conn.commit()
    print("Database cleared.")
```

- `SUM(price)` over an empty table returns `None` (not 0). `fetchone()[0] or 0` turns `None` into 0. Without it, `Display cost` after Clear All would print "None".
- `DELETE FROM Passengers` with **no WHERE** deletes every row (but keeps the table). Analogy: tear out every page but keep the ticket book.
- `f"{age:>21}"`: right-align in 21 characters, so columns line up.

4. If it is a window again (tkinter map)

Task	tkinter code in the window solution
read the age	`age_entry.get()`
clear it	`age_entry.delete(0, tk.END)`
fill a list window	`age_list.insert(tk.END, age)` ; clear with `age_list.delete(0, tk.END)`
show the total	`cost_box.insert(tk.END, str(total))`
a bad age	`messagebox.showerror("Invalid age", "...")`

5. Traps

- **TRAP:** storing only the age and recomputing the price at display time works, but the paper says the price must be **saved in the database**: store both.
- **TRAP:** `SELECT SUM(price)` returns a tuple `(44,)`: use `[0]`.
- **TRAP:** forgetting `ORDER BY id`: SQLite usually returns insertion order, but without it nothing guarantees ages and prices come back in the same order.
- **TRAP:** Clear All that only clears the screen. The paper says "delete everything from the database" (restart the program and nothing should appear): you need `DELETE FROM` and `commit()`.
- **TRAP:** treating age 65 or 66 wrongly. "Seniors aged 66 and over": 66 pays £13, 65 pays £18.

6. Anticipated questions

1. *Add a "family discount": 10% off the total if there are 4 or more passengers.* `if count >= 4: total *= 0.9`; get count with `SELECT COUNT(*)`.
2. *Show the average age.* `SELECT AVG(age) FROM Passengers`.
3. *Show the oldest and youngest passengers.* `SELECT MIN(age), MAX(age) FROM Passengers`.
4. *How many passengers pay each price?* `SELECT price, COUNT(*) FROM Passengers GROUP BY price`.
5. *Reject ages above 120 or below 0.* The range check already in the solution.
6. *Change prices without editing four places.* Keep them in a dictionary at the top: `PRICES = {"free": 0, "child": 7, "senior": 13, "adult": 18}`.
7. *What happens if the database file already holds old data when the program starts?* It persists; that is why Clear All exists.

7. Quick-reference box

```
fare      if age <= 2: 0 / elif age <= 12: 7 / elif age >= 66: 13 / else: 18
loop      while True: t = input(); if t.lower() == "q": break
validate  try: age = int(t) / except ValueError: print(...); continue
total     cur.execute("SELECT SUM(price) FROM T"); total = cur.fetchone()[0] or 0
clear all cur.execute("DELETE FROM T"); conn.commit()
```